Lab 4: Simple Python Database Connections

Solution Document

Ubuntu identity	sadia612@ubuntu
GitHub username	Sadia612
Databases fork URL	https://github.com/Sadia612/Databases
Forked from	https://github.com/WaqasSaleem97/Databases
Submission directory	DevOps/Labs/Lab04
Environment	Ubuntu server; Python 3 in .venv; Docker runs MySQL 8.4 and MongoDB 8.0

Summary of Results

Stage	Database	Port	Environment value
Task 4: values in code	school_mysql (MySQL)	3307	dev
Task 5: values in code	school_dev (MongoDB)	27017	dev
Task 6: os.getenv, dev	school_dev	27017	dev
Task 6: os.getenv, test	school_test	27018	test
Task 6: os.getenv, prod	school_prod	27019	prod
Task 7: credentials file, dev	school_dev	27017	dev
Task 7: credentials file, test	school_test	27018	test
Task 7: credentials file, prod	school_prod	27019	prod

Every connection was confirmed by three things: the database name printed by the program, the server port, and the environment value read from the actual database (environment_info). Python ran directly on Ubuntu and Docker ran only the databases.

Task 1: Fork and Clone the Databases Repository

I forked the instructor's Databases repository to my own account (Sadia612/Databases) and cloned my fork over SSH to the Ubuntu server. The clone contains db_connect.py, compose.yaml, requirements.txt, credentials.ini.example and db-init.

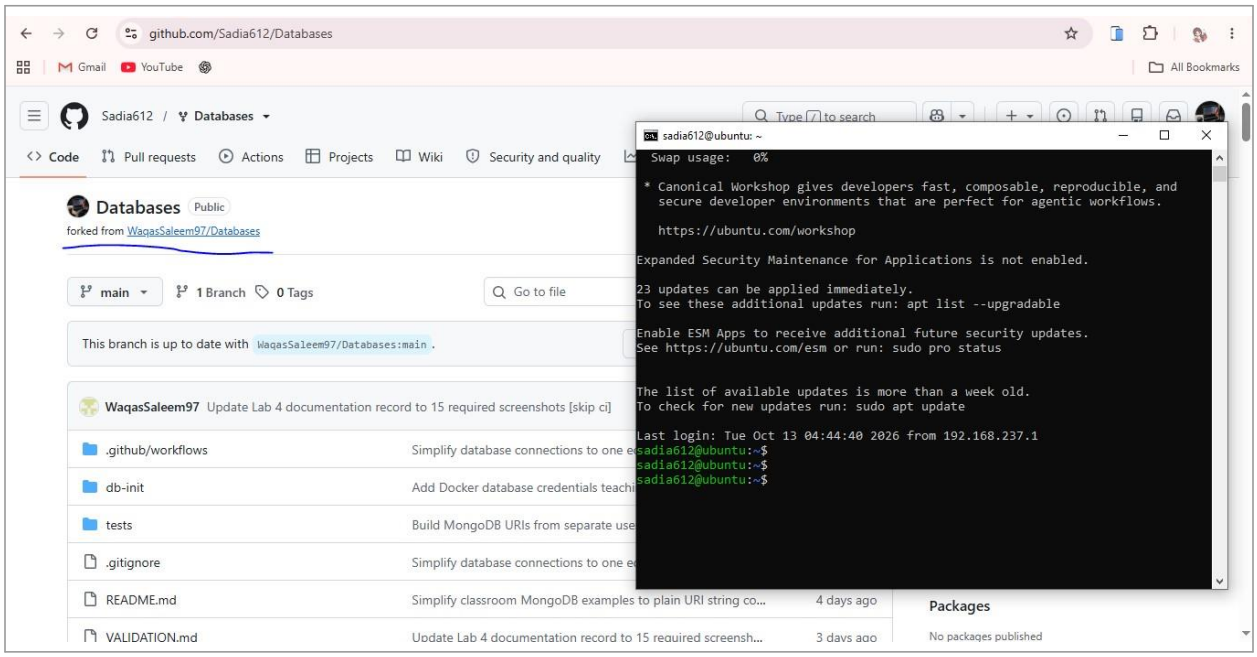


Figure: My fork on GitHub, shown as 'forked from WaqasSaleem97/Databases', with the Ubuntu terminal beside it (databases_fork.png)

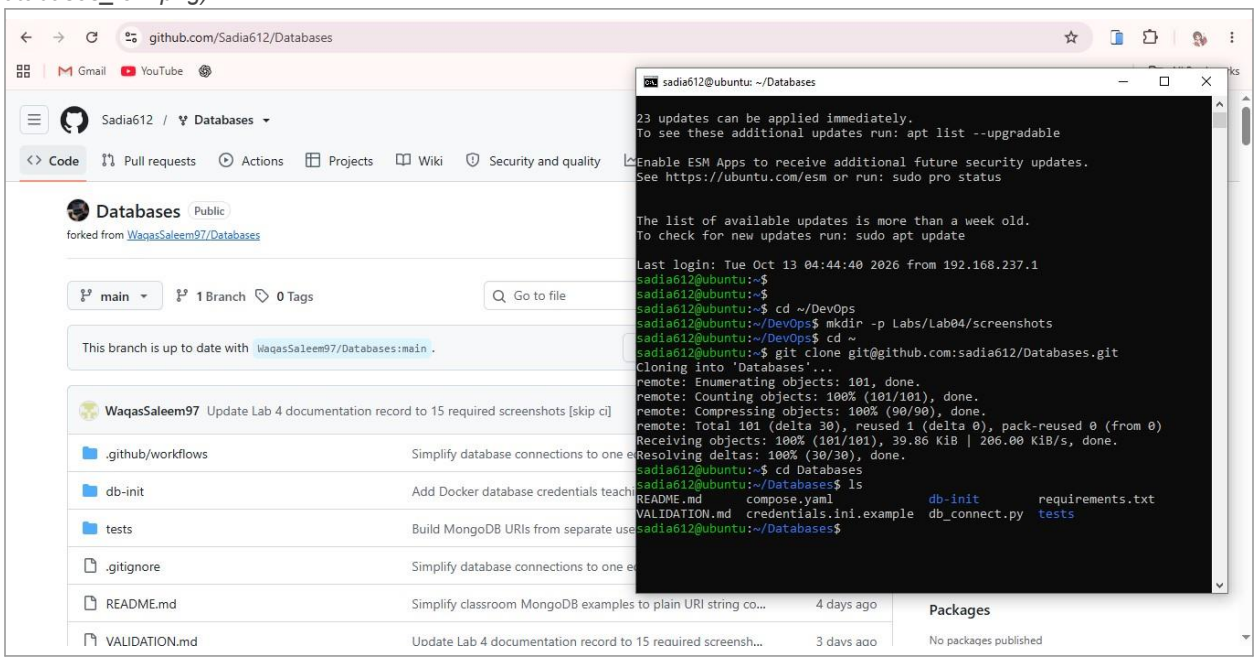


Figure: git clone of my fork and ls of the cloned directory (databases_clone.png)

Task 2: Install Docker, Docker Compose and Python

I installed docker.io and curl with apt, downloaded the Docker Compose plug-in, started Docker and added my account to the docker group, then signed out and back in. After reconnecting, Git 2.53.0, Docker 29.1.3

and Docker Compose v5.6.1 were all installed. I then created a virtual environment in ~/Databases and installed pymongo, mysql-connector-python and dnspython from requirements.txt.

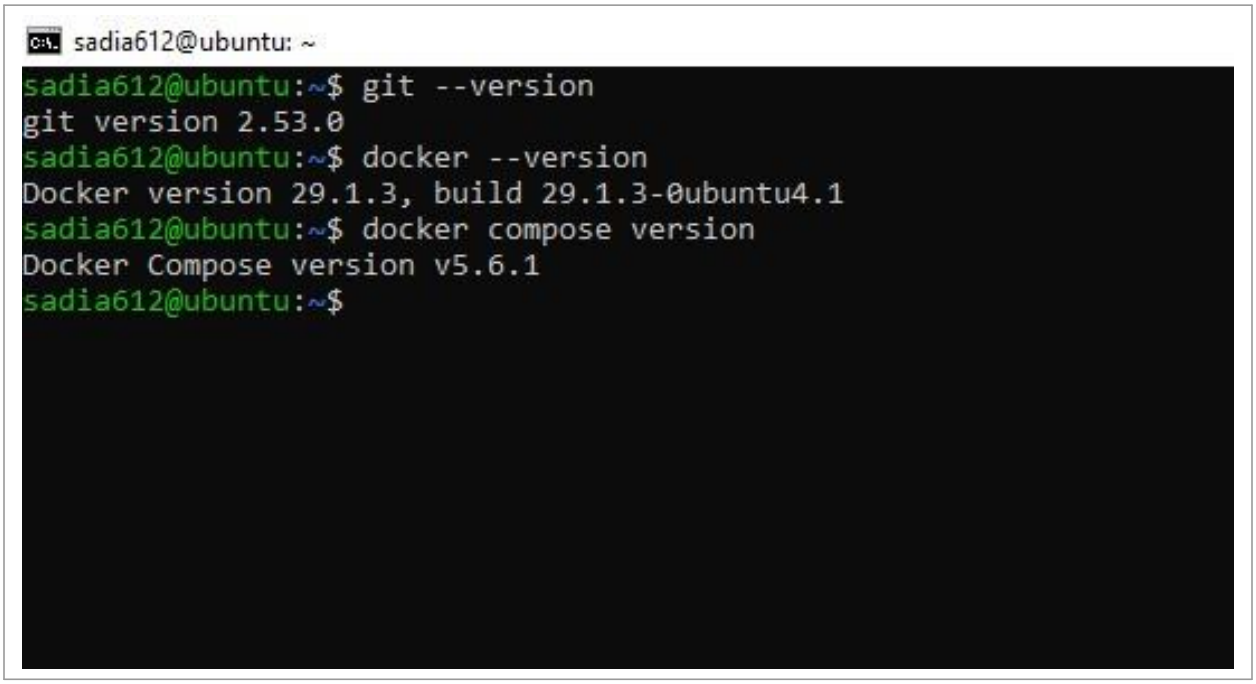


Figure: git, docker and docker compose versions (docker_installation.png)

Task 3: Start the Database Services

I started the four databases with `docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod`. All four services reached the healthy state and use the host endpoints below.

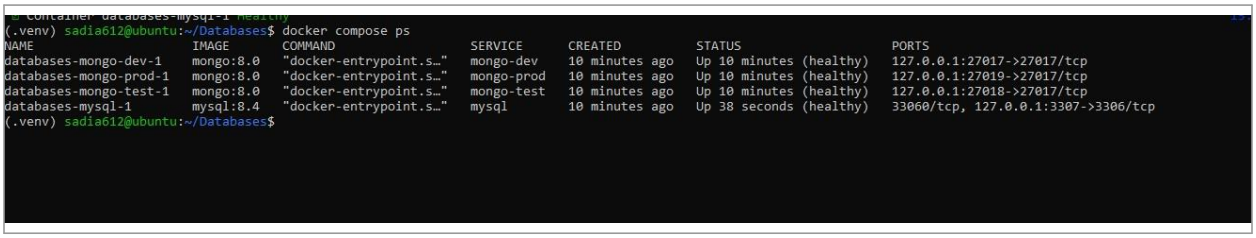


Figure: docker compose ps showing all four services healthy (database_services.png)

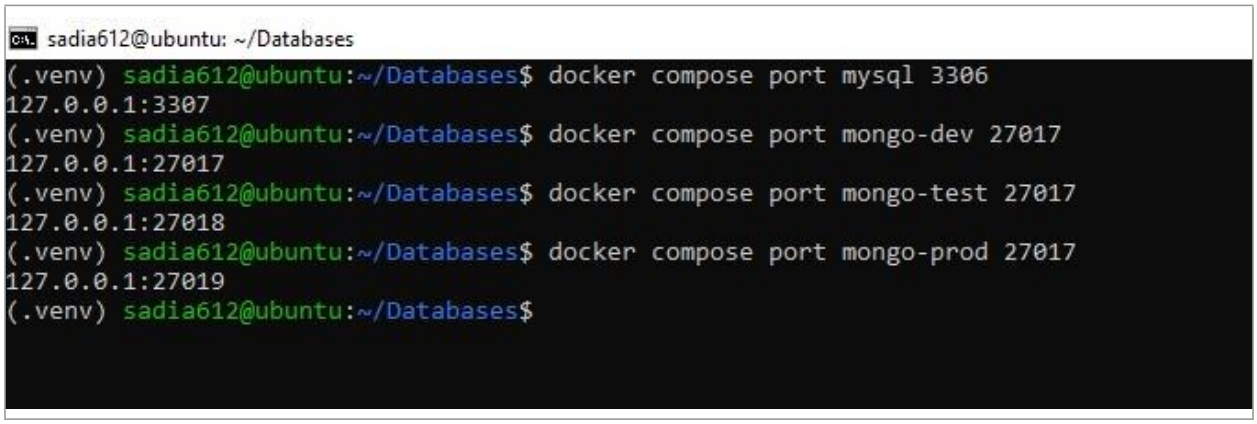


Figure: docker compose port output for the four services (database_ports.png)

Problems encountered and how they were solved

1. **MongoDB containers exited immediately.** The logs said that Linux kernel 6.19 and newer has a known incompatibility with this MongoDB version (MongoDB issue SERVER-121912). My Ubuntu server uses a newer kernel. I confirmed the CPU supports AVX, so that was not the cause. As a workaround I added a `compose.override.yaml` file that sets the environment variable `GLIBC_TUNABLES=glibc.pthread.rseq=1` for mongo-dev, mongo-test and mongo-prod. The supplied compose.yaml was not modified. After running `docker compose down -v` and starting again, all MongoDB services became healthy.

2. **MySQL exited with code 137.** The server has only 1.6 GB of RAM, and starting four databases at once made the system stop MySQL because memory ran low. I restarted it with `docker compose up -d --wait mysql` and it became healthy and stayed up.

Database	Host endpoint	Database name
MySQL	127.0.0.1:3307	school_mysql
MongoDB dev	127.0.0.1:27017	school_dev
MongoDB test	127.0.0.1:27018	school_test
MongoDB simulated prod	127.0.0.1:27019	school_prod

Task 4: MySQL with Values in the Code

The starter db_connect.py connects to MySQL with mysql.connector.connect(). The user and password supply the credentials, host and port identify the server and its MySQL service, and database selects the database. The program then runs a small query that reads the environment value stored in MySQL, and closes the cursor and the connection to release resources.

Result: Connected to MySQL: school_mysql, Server: 127.0.0.1 3307, Environment: dev.

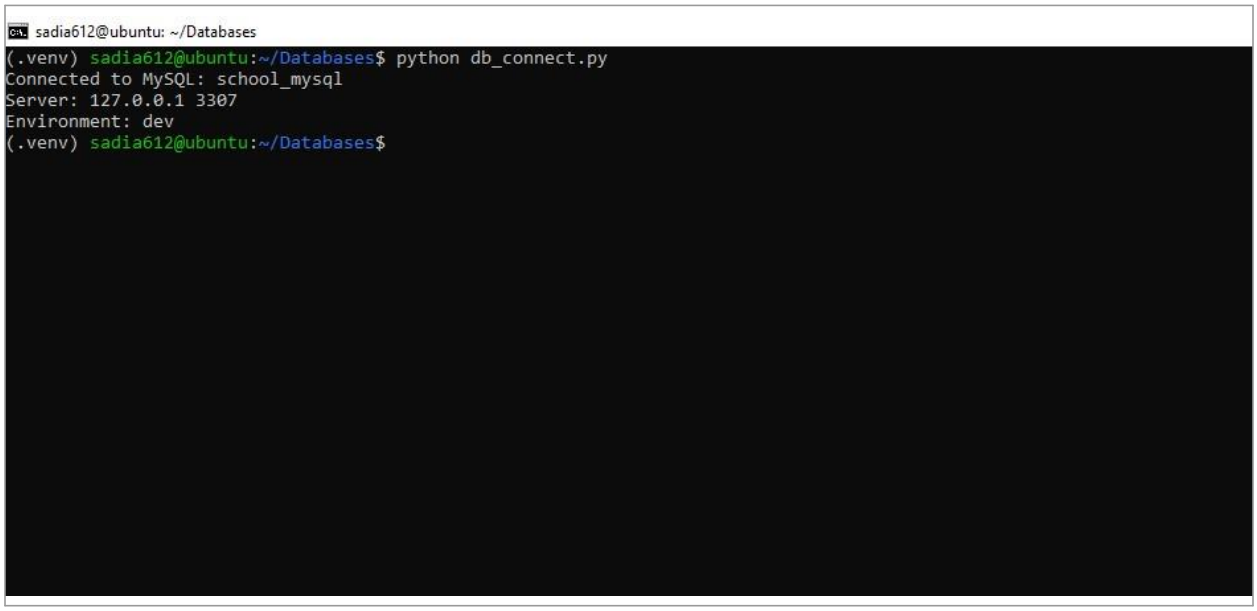


Figure: MySQL connection with values written in the code (mysql_hardcoded_connection.png)

Task 5: MongoDB with Values in the Code

I replaced the contents of the same db_connect.py. Four variables (username, password, endpoint, database_name) hold the settings separately, and the + operator joins them into the connection URI. MongoClient(uri) creates the client and client[database_name] selects the database. `database.command('ping')` sends a real request, because creating a client alone does not prove connectivity. serverSelectionTimeoutMS=5000 limits server selection to five seconds. The URI's database is also the authentication database in this lab.

Result: database school_dev, port 27017, environment dev.

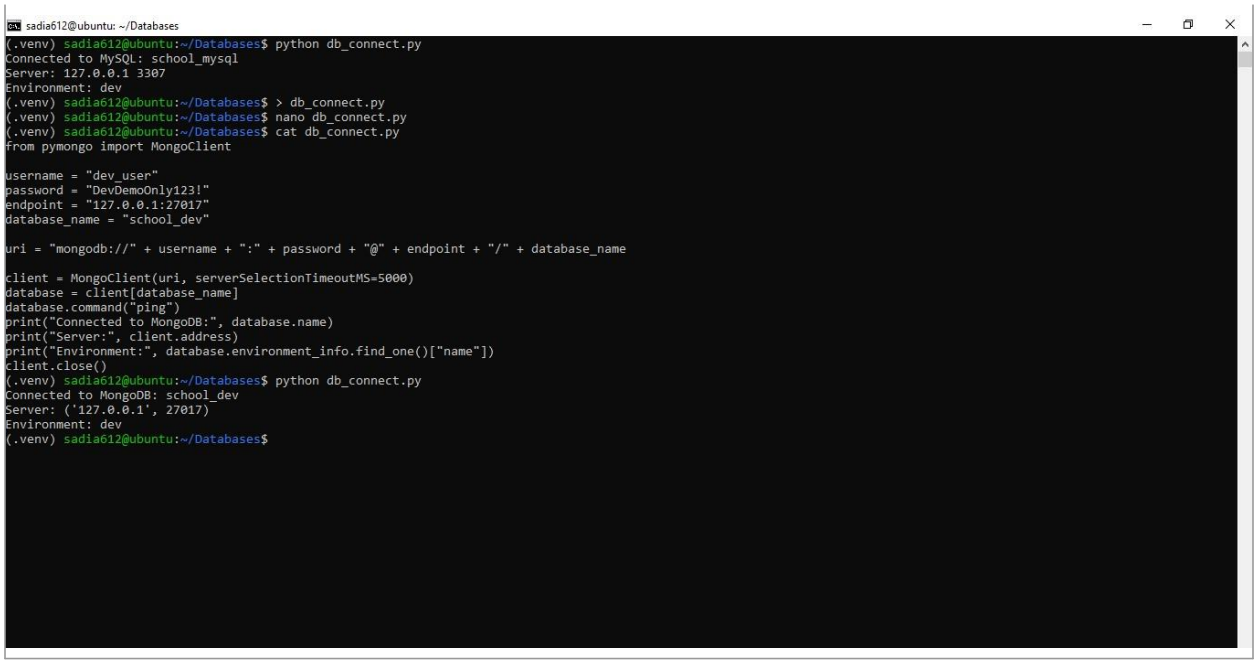


Figure: MongoDB connection with values written in the code (mongo_hardcoded_connection.png)

Task 6: MongoDB with os.getenv()

The same file now reads MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT and MONGO_DATABASE with os.getenv(). A short if statement stops the program with a message if any variable is missing. The URI concatenation, MongoDB connection and output block were left unchanged. Python inherits the exported variables from the shell, so the same unchanged program connected to three different deployments just by changing the four exported values. This version does not read APP_ENV.

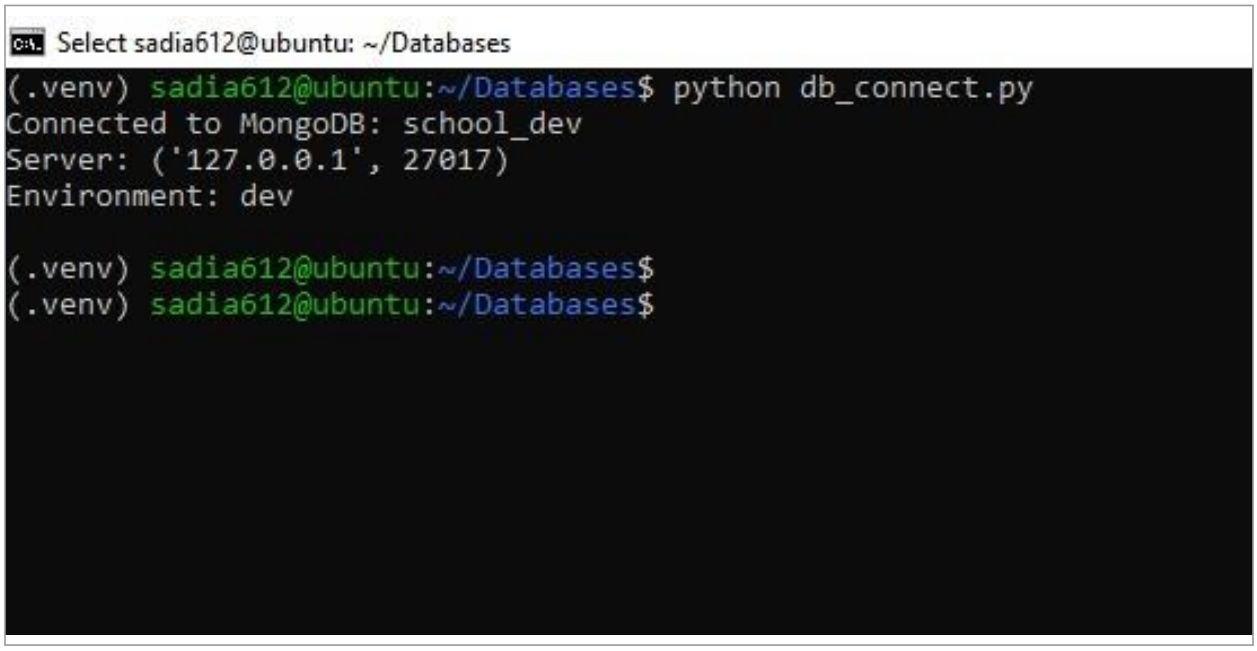


Figure: os.getenv version against dev (school_dev, port 27017, environment dev) (mongo_env_dev.png)

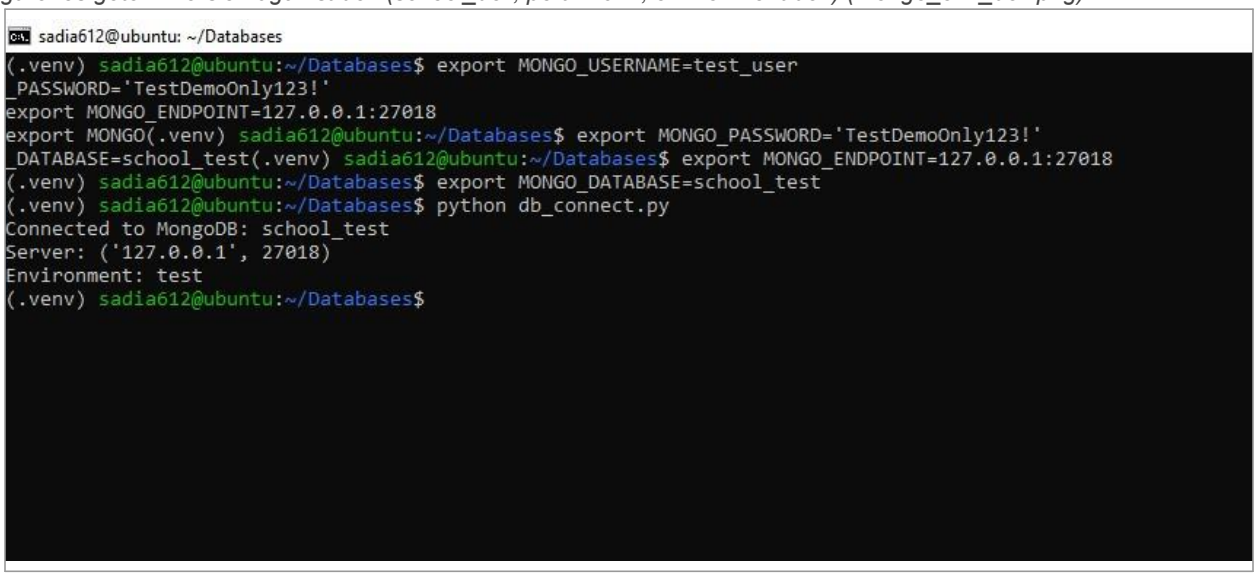


Figure: os.getenv version against test (school_test, port 27018, environment test) (mongo_env_test.png)

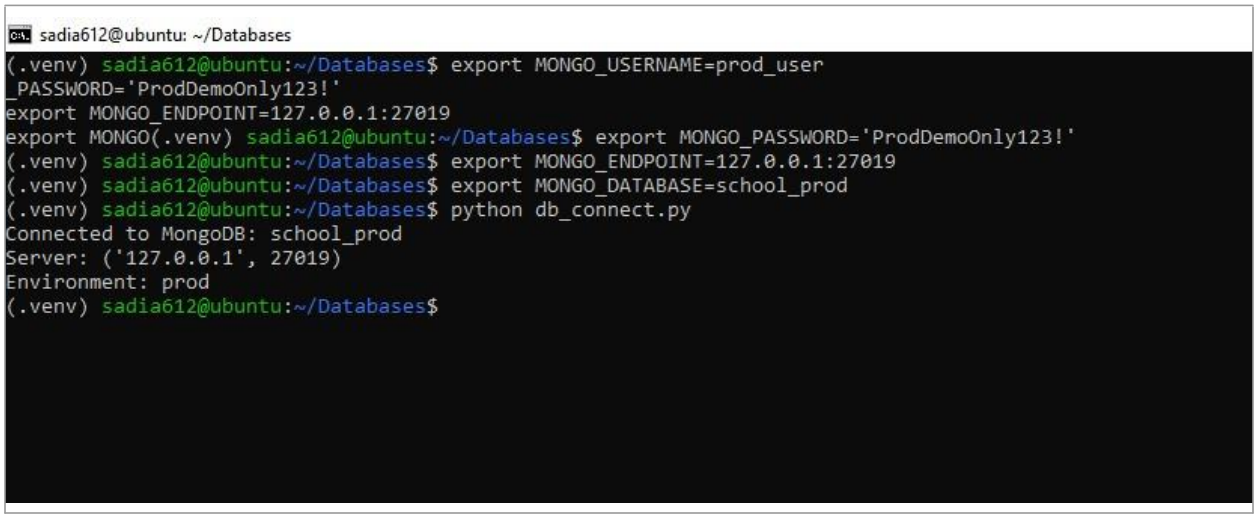


Figure: os.getenv version against simulated prod (school_prod, port 27019, environment prod) (mongo_env_prod.png)

Task 7: MongoDB with a Credentials File

I copied credentials.ini.example to credentials.ini, which holds three sections (dev, test, prod) with username, password, endpoint and database. The file is ignored by Git, so it is not committed. I restricted it to my account with chmod 600 and verified the mode is 600.

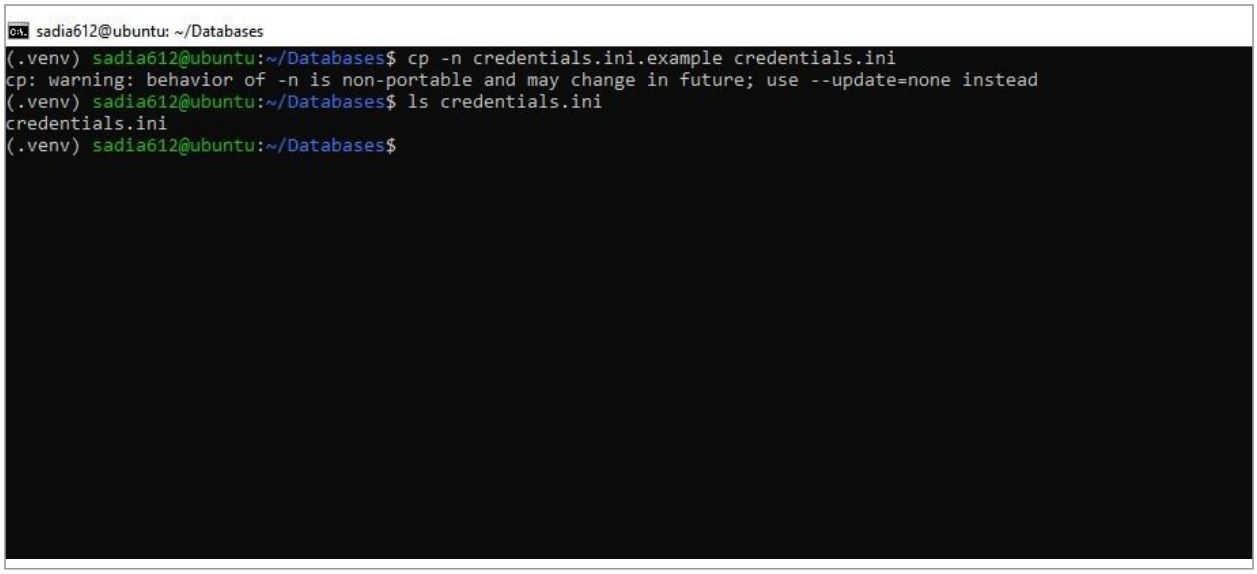


Figure: credentials.ini created from the example file (credential_files_created.png)

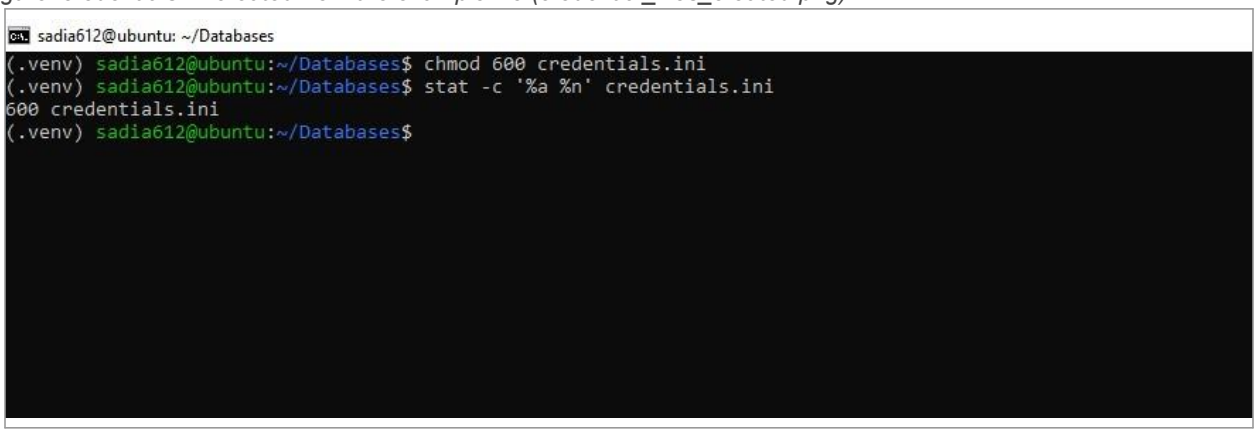


Figure: credentials.ini restricted to mode 600 (credential_file_permissions.png)

The final db_connect.py uses configparser. config.read() loads the file, `os.getenv('APP_ENV', 'dev')` picks the section name (dev by default), and config[environment] supplies the four settings. interpolation=None makes the reader treat values literally. The connection block is the same as in the earlier stages.

```
import os import configparser
from pymongo import
MongoClient

config =
configparser.ConfigParser(interpolation=None)
config.read("credentials.ini") environment =
os.getenv("APP_ENV", "dev") username =
config[environment]["username"] password =
```

```
    config[environment]["password"] endpoint =
    config[environment]["endpoint"]
```

```
    database_name = config[environment]["database"]

uri = "mongodb://" + username + ":" + password + "@" + endpoint + "/" + database_name

    client = MongoClient(uri, serverSelectionTimeoutMS=5000) database
    = client[database_name] database.command("ping") print("Connected
    to MongoDB:", database.name) print("Server:", client.address)
    print("Environment:",
    database.environment_info.find_one()["name"]) client.close()
```

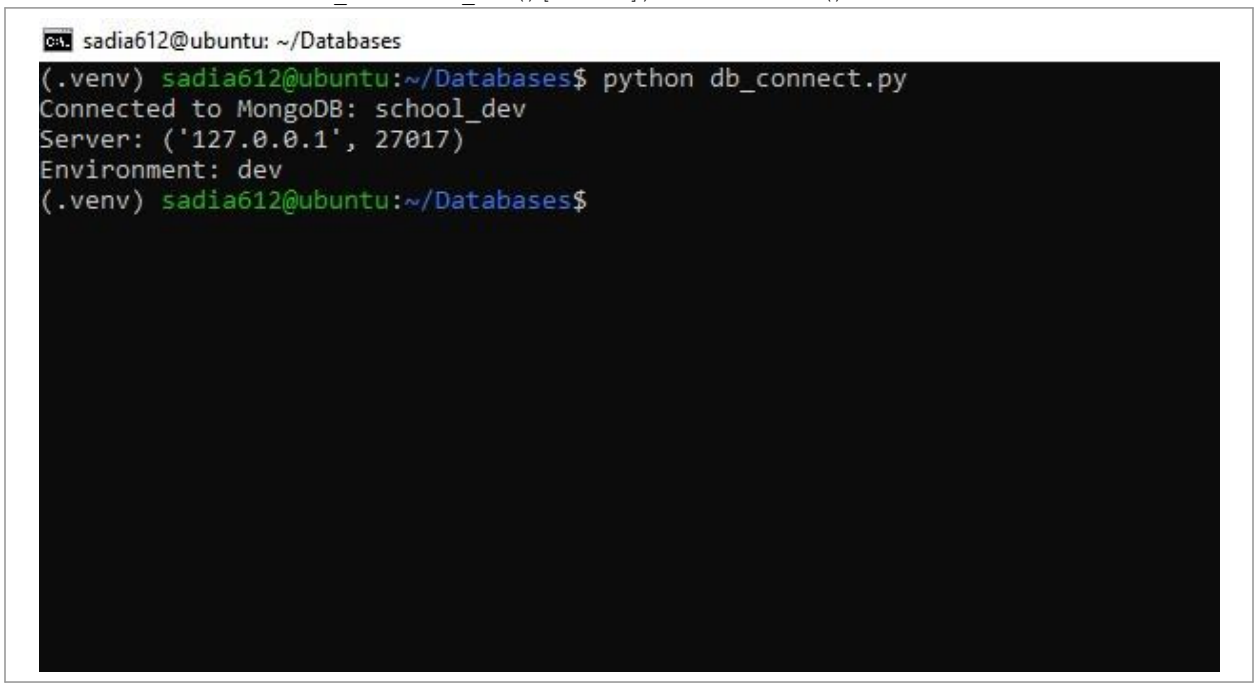


Figure: credentials-file version with APP_ENV=dev (school_dev, port 27017) (mongo_file_dev.png)

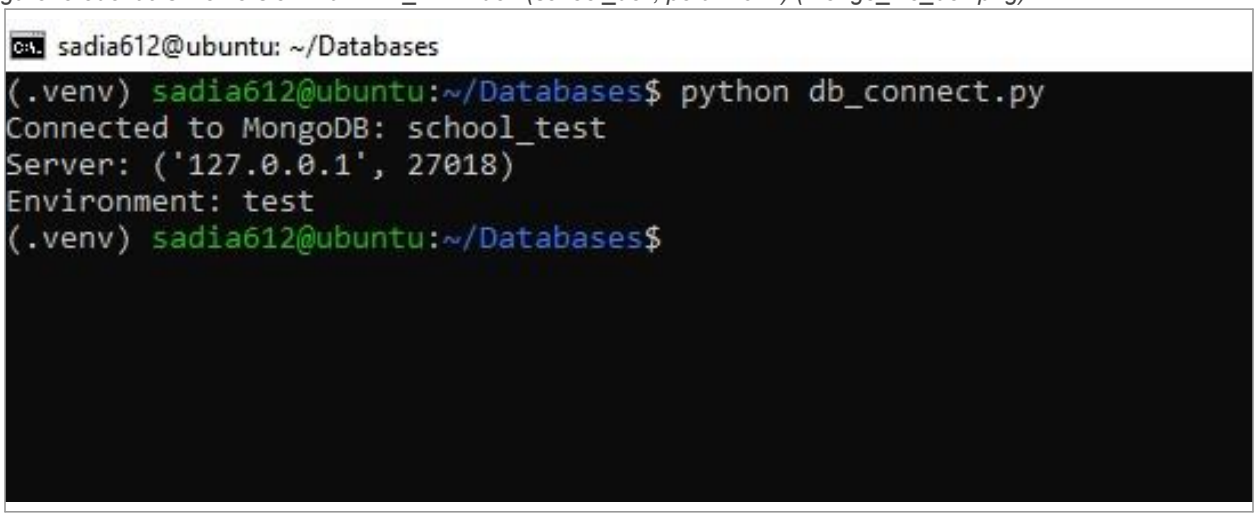


Figure: credentials-file version with APP_ENV=test (school_test, port 27018) (mongo_file_test.png)

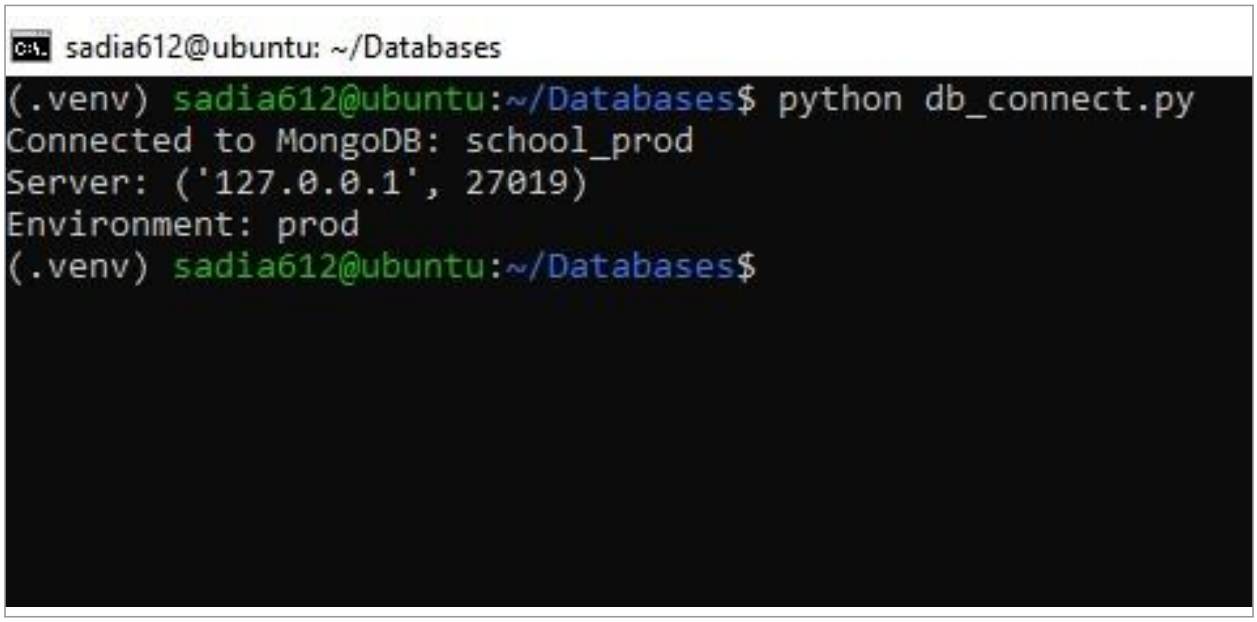


Figure: credentials-file version with APP_ENV=prod (school_prod, port 27019) (mongo_file_prod.png)

Exam Evaluation Questions

1. A Missing Environment Variable

With the prod values exported, I ran `unset MONGO_DATABASE` and then `python db_connect.py`. The program printed 'Set all four MongoDB environment variables first.' and did not connect.



Figure: Program stops when MONGO_DATABASE is missing (Q1_missing_environment.png)

Explanation: The call `os.getenv('MONGO_DATABASE')` returned None, because that variable no longer existed in the shell environment. Since database_name was None, the condition `not database_name` became true, so the if statement ran SystemExit with the message. The program therefore stopped before it built the URI or created any MongoClient. The other three variables were still set; one missing value is enough.

I then restored `export MONGO_DATABASE=school_prod` and the program connected to school_prod on port 27019 again.

2. Which Source Does the File Version Read?

With the prod MONGO_* variables from Task 6 still exported, I ran `unset APP_ENV` and then ran the program. It connected to dev.

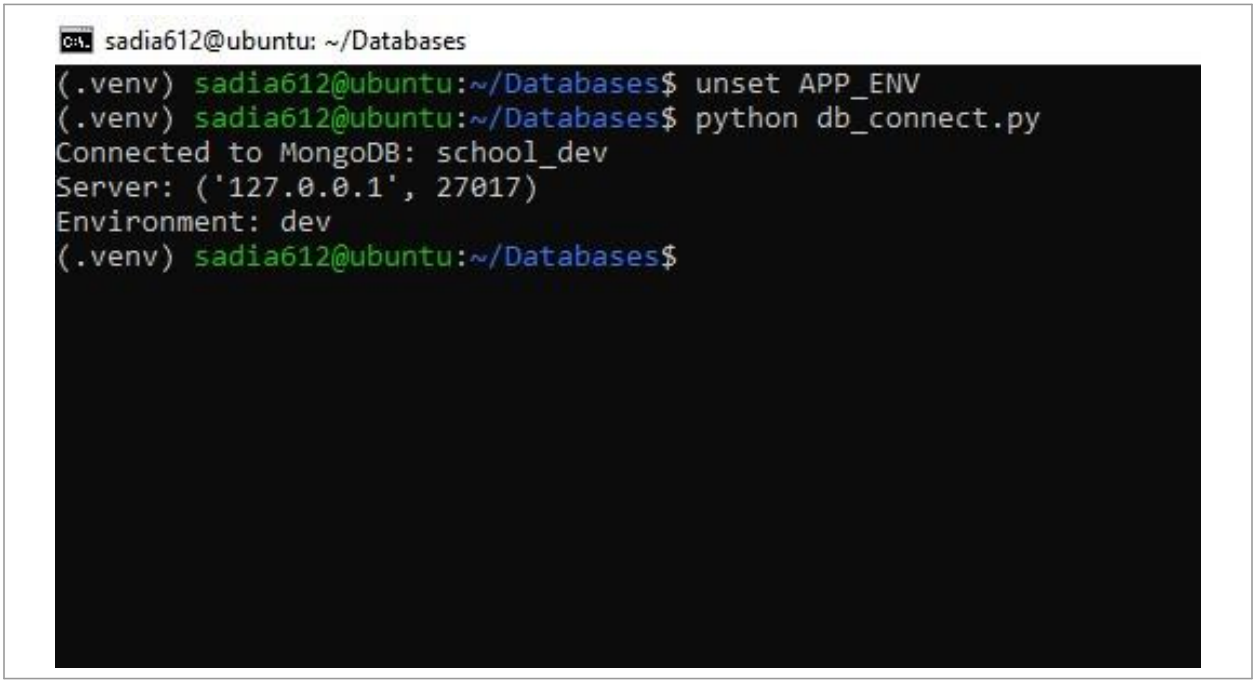


Figure: File version connects to dev when APP_ENV is unset (Q2_file_source_selection.png)

Explanation: The credentials-file version never reads MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT or MONGO_DATABASE. Its only input from the environment is APP_ENV, which chooses the section of credentials.ini. When APP_ENV is unset, os.getenv('APP_ENV', 'dev') returns its default value dev, so the [dev] section is used (school_dev, port 27017). The prod variables still in the shell are simply ignored. Selecting APP_ENV=test makes the program use the [test] section and connect to school_test on port 27018, as shown in mongo_file_test.png.

3. A Wrong Password in the File

In the [test] section of my local credentials.ini only, I temporarily set the password to 'wrong', kept the username, endpoint and database name unchanged, and ran the program with APP_ENV=test. The connection failed with an authentication error raised by pymongo when database.command('ping') made its first real request.

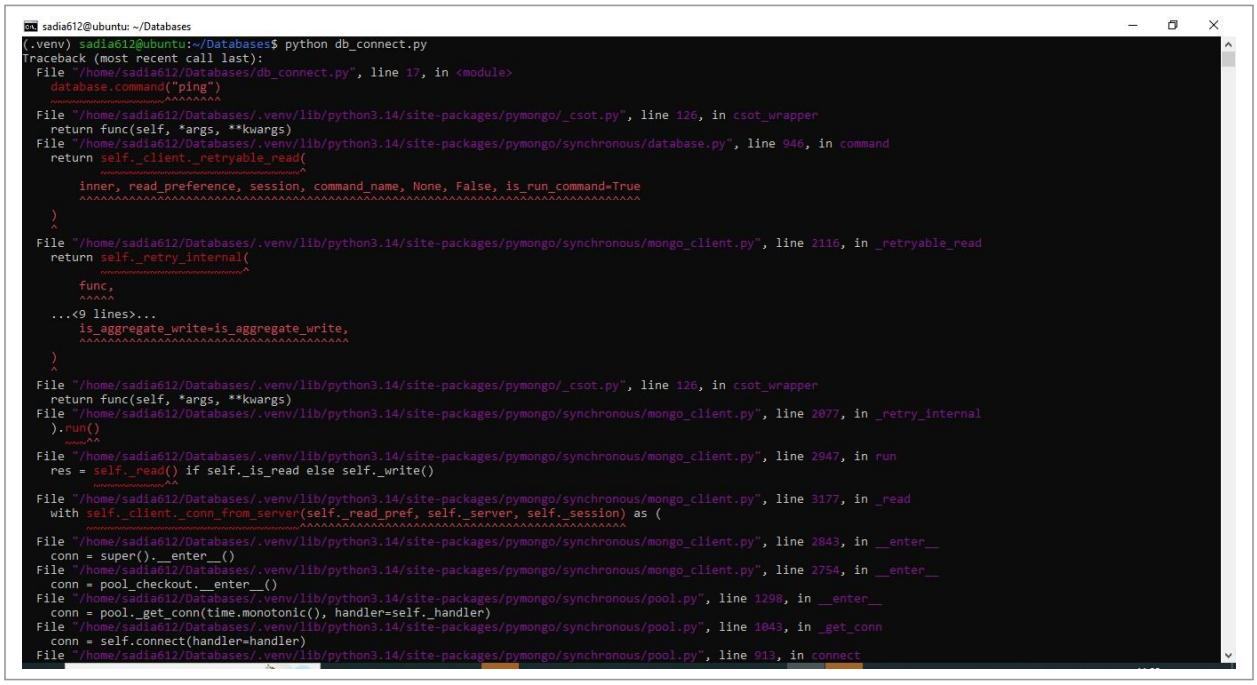


Figure: Authentication failure with the wrong password in the [test] section (Q3_wrong_file_password.png)

Explanation: The endpoint only tells Python which server and port to reach, and that part worked because the server was running and reachable. After the connection is opened, MongoDB separately checks the username and password against the stored user in the authentication database. A wrong password fails that check regardless of whether the endpoint is correct. Finding the right server is not the same as being allowed to use it.

I then restored the supplied test dummy password, ran the program again successfully, and set APP_ENV=prod. credentials.ini is ignored by Git and was not committed.